

Design Justification Report: INT8 2D Convolution Accelerator

ECE 410/510 HW4AI — Milestone 4 — Spring 2026

Karthik Bandla

June 7, 2026

1. Problem and Motivation

The target kernel is a 2D convolution layer from a YOLO-style object detection network. The input is a $64 \times 64 \times 3$ INT8 feature map, convolved with 16 filters of size $3 \times 3 \times 3$ (stride 1, same-padding), producing a $64 \times 64 \times 16$ INT8 output feature map. This single layer performs 1,769,472 multiply-accumulate operations (MACs) and 3,538,944 floating-point-equivalent operations (FLOPs) per inference.

CPU profiling was conducted in M1 on an AMD Ryzen 7 7730U using Python 3.13.0 and NumPy 2.4.4, with $N=1000$ repeated measurements. The median runtime was **1,257.30 μ s per inference** (mean 1,261.60 μ s, std 44.98 μ s), corresponding to 795 inferences/sec and 2.815 GFLOP/s. This single convolution layer accounts for 99.2% of the total YOLO pipeline runtime (conv $\rightarrow$ ReLU $\rightarrow$ max pool), making it the dominant bottleneck. Despite the CPU having a theoretical peak of 72 GFLOP/s, the measured utilization is only 3.91%, indicating that the CPU cannot effectively vectorize this workload without specialized INT8 kernels.

Custom hardware acceleration is justified by three factors. First, the workload is fixed-function: the convolution kernel geometry ($3 \times 3 \times 3$, stride 1, same-padding) is structurally invariant across all inference calls, enabling full circuit specialization without the overhead of a general-purpose processor. Second, the 4,096 output positions (64×64) are fully independent and perfectly parallelizable — there are no data dependencies between spatial positions, making this an ideal target for a spatial compute array. Third, edge deployment scenarios (IoT cameras, robotics, autonomous vehicles) require power budgets of milliwatts, not the ~ 15 W TDP of the CPU. The M1 profiling showed that energy per inference on the CPU is approximately 18,855 μ J, which is unsustainable for battery-powered operation.

The system-level architecture is shown in Figure 1 (report/figures/system_diagram.png), which illustrates the AXI4-Lite control path, the AXI4-Stream data path, the weight register file, line buffer, MAC array, and output pipeline.

2. Roofline Analysis

The roofline model provides a principled framework for understanding the performance ceiling of any compute kernel on any hardware platform. It plots achievable performance (GFLOP/s) against arithmetic intensity (FLOP/byte) on log-log axes, bounded above by two ceilings: the peak compute rate and the peak memory bandwidth times the arithmetic intensity. The roofline for both the CPU baseline and the M4 accelerator is shown in Figure 2 (bench/roofline_final.png).

Kernel arithmetic intensity. The convolution layer accesses 12,288 bytes of input, 432 bytes of weights, and produces 65,536 bytes of output. Under a weight-stationary dataflow where weights are loaded once and reused across all 4,096 positions, and input pixels are reused via the 3-row line buffer (each pixel is used in up to 9 different 3×3 windows), the effective arithmetic intensity is $3,538,944 \text{ FLOPs} / (12,288 + 432 + 65,536) \text{ bytes} = \mathbf{45.19 \text{ FLOP/byte}}$. This places the kernel deep in the compute-bound region for both platforms.

CPU roofline. The AMD Ryzen 7 7730U has a theoretical peak single-core compute of 72.0 GFLOP/s and peak memory bandwidth of 51.2 GB/s (DDR4-3200 dual channel). The ridge point (compute/bandwidth) is at $72.0/51.2 = 1.41 \text{ FLOP/byte}$. Since $45.19 \gg 1.41$, the kernel is firmly compute-bound on the CPU. The theoretical ceiling at $\text{AI}=45.19$ is 72 GFLOP/s, but the measured performance is only 2.815 GFLOP/s — 3.91% of peak. This gap is explained by the CPU’s inability to issue vectorized INT8 multiply-accumulate operations without explicit AVX-512 VNNI kernels; NumPy falls back to scalar or FP32 computation paths.

Accelerator roofline. The M4 accelerator has a peak compute rate of $16 \text{ MACs} \times 100 \text{ MHz} \times 2 \text{ FLOP/MAC} = \mathbf{3.2 \text{ GFLOP/s}}$. The AXI4-Stream interface at 32-bit width and 100 MHz provides a peak bandwidth of 400 MB/s = $\mathbf{0.4 \text{ GB/s}}$. The ridge point is at $3.2/0.4 = 8.0 \text{ FLOP/byte}$. Since $45.19 \gg 8.0$, the accelerator is also compute-bound, which means the interface is never the bottleneck regardless of how fast pixels arrive. The measured performance is $\mathbf{2.606 \text{ GFLOP/s}}$ (81.4% of the 3.2 GFLOP/s peak), with the gap accounted for by FSM overhead cycles between consecutive MAC windows.

The roofline analysis shaped two key architectural decisions. First, it confirmed that weight-stationary dataflow was the correct choice: any dataflow that requires re-fetching weights from off-chip memory would move the kernel toward the memory-bound regime, reducing performance. Second, it justified the 3-row line buffer design: without it, each input pixel would be fetched once per output position that uses it (up to 9 times), increasing effective data movement by $9\times$ and dropping the arithmetic intensity by the same factor, potentially making the design memory-bound.

3. Precision and Data Format

Selected format: INT8 inputs and weights, INT32 accumulator, INT8 output (post-ReLU requantization).

INT8 was selected for this design rather than FP16 or FP32 for four reasons. First, memory capacity: $16 \text{ filters} \times 27 \text{ taps} \times 8 \text{ bits} = 432 \text{ bytes}$ fits comfortably in on-chip flip-flop registers, whereas FP32 weights would require 1,728 bytes — a $4\times$ increase that would push the design into SRAM territory with significantly higher area and

power overhead. Second, area efficiency: a signed 8×8-bit multiplier in sky130A requires approximately 200 μm^2 , while a 32-bit floating-point multiplier requires approximately 5,000 μm^2 — a 25× area penalty that would make 16 parallel MACs infeasible within the area budget. Third, accumulator safety: with 27 taps, each with INT8 inputs in the range [-128, 127], the worst-case accumulation magnitude is $27 \times 127 \times 128 = 438,912$, which fits comfortably in a 20-bit signed integer. The 32-bit accumulator provides a 12-bit safety margin above this worst case, so no overflow is possible. Fourth, YOLO-style network accuracy: INT8 quantization with symmetric per-tensor scaling ($S = \max(|W|)/127$) introduces less than 1% mAP degradation on typical YOLO detection benchmarks, making the precision-accuracy tradeoff acceptable.

The requantization step at the output pipeline (`output_pipeline.sv`) applies a right-shift of 8 bits to the 32-bit accumulator, implementing an effective scaling factor of 1/256. This is followed by a ReLU activation (clamp to zero for negative values) and saturation clamping to [0, 127] for the INT8 output. The mathematical correctness of this scheme was verified in M2 using a Python reference model that reproduced the expected output values at three spatial positions: $\text{pos}(0,0)=1$, $\text{pos}(0,1)=0$, $\text{pos}(1,0)=2$, all matching the hardware output exactly.

All arithmetic operands in `compute_core.sv` are declared signed, and all products are explicitly sign-extended to 32 bits before accumulation. This avoids the Verilog self-determined context truncation bug (where an unsigned context would silently zero-extend rather than sign-extend) that was identified during M2 debugging.

4. Dataflow and Architecture

Selected dataflow: Weight-stationary.

In weight-stationary dataflow, each filter's complete set of weights is loaded into on-chip registers before inference begins and remains stationary throughout the entire computation. All 4,096 spatial output positions are computed with the same set of weights, giving a weight reuse factor of 4,096×. This is the dominant factor in the kernel's arithmetic intensity of 45.19 FLOP/byte.

The alternative dataflows considered were output-stationary (accumulate partial sums in a fixed output buffer, re-fetch weights each time) and input-stationary (hold input activations in place, sweep weights over them). Output-stationary was rejected because it requires fetching $16 \times 27 = 432$ weights for every output position, giving zero weight reuse and dropping the arithmetic intensity to approximately 0.5 FLOP/byte — squarely memory-bound. Input-stationary was rejected because the 3-row line buffer already provides high input reuse (each pixel reused up to 9 times), and combining this with input-stationary would require a much larger on-chip buffer to hold all input activations simultaneously (12,288 bytes vs 576 bytes for 3 rows).

The complete data path is as follows. Input pixels arrive over the AXI4-Stream slave port at 8 bits per beat. The control FSM accepts pixels when in the FILL state (`pixel_accept=1`), writing them to the 3-row circular line buffer (`line_buffer.sv`) in row-major, channel-interleaved order. When the line buffer has accumulated enough rows

to form a valid $3 \times 3 \times 3$ window at the current output position, it asserts `window_valid`, and the FSM transitions to `WIN_LATCH`, capturing the flattened 27-element window. The FSM then enters `MAC_RUN` for 27 cycles: on each cycle, it presents one (pixel, weight) pair to all 16 `compute_core` instances simultaneously. After 27 cycles, the FSM waits one cycle for accumulator settling (`ACC_SETTLE`), then one cycle for the output pipeline to process the results and emit a 128-bit output beat ($16 \times \text{INT8}$, one per filter) on the AXI4-Stream master port. After the streaming rows are processed, the FSM enters `FLUSH` mode to process the final rows using internally generated zero-padded flush windows. The architecture is illustrated in Figure 1 (report/figures/algorithm_diagram.png).

5. Hardware Interface

Selected interface: AXI4-Lite (control plane) + AXI4-Stream (data plane).

This interface combination was selected in M1 based on three criteria: compatibility with the Xilinx Zynq PS-PL interface (AXI is the native interconnect), backpressure support for flow control (TVALID/TREADY handshake), and separation of control and data paths (AXI4-Lite for register-mapped configuration, AXI4-Stream for high-throughput pixel data).

AXI4-Lite register map:

- 0x00 CTRL: bit0=start, bit1=wt_done
- 0x04 STATUS: bit0=busy, bit1=done
- 0x14 WEIGHT_ADDR: filter[4:0] | tap[4:0] (encodes which weight to load)
- 0x18 WEIGHT_DATA: 32-bit sign-extended INT8 weight value

The host loads all $16 \times 27 = 432$ weights via sequential AXI4-Lite writes to `WEIGHT_ADDR` and `WEIGHT_DATA`, then writes `CTRL=0x3` (wt_done=1, start=1) to begin inference.

AXI4-Stream data path:

- Slave port: 8-bit TDATA, one INT8 pixel per beat, 12,288 beats per inference
- Master port: 128-bit TDATA, 16 INT8 output values per beat, 4,096 beats per inference, TLAST asserted on the final beat

Effective bandwidth analysis:

- Required input bandwidth: $12,288 \text{ bytes} / 1,357 \mu\text{s} = 9.1 \text{ MB/s}$
- Required output bandwidth: $65,536 \text{ bytes} / 1,357 \mu\text{s} = 48.3 \text{ MB/s}$
- AXI4-Stream capacity at 32-bit/100 MHz: 400 MB/s

The input interface provides $44\times$ more bandwidth than required, and the output interface provides $8\times$ more bandwidth than required. The design is therefore definitively compute-bound and not interface-bound, consistent with the roofline analysis in Section 2.

Alternatives that were considered and rejected include SPI (maximum $\sim 6 \text{ MB/s}$, $1,500\times$ insufficient for real-time inference), I2C (maximum $\sim 0.4 \text{ MB/s}$), and a simple register-only interface (would require 12,288 sequential CPU writes to load pixels, adding $\sim 30 \mu\text{s}$ of software overhead per inference).

6. Verification

A layered verification strategy was used across milestones M2, M3, and M4, progressing from unit-level tests to full system simulation with numerical verification against a software reference.

Unit test (M2, compute_core, cocotb). The cocotb testbench `test_conv2d_core.py` applied a reset sequence followed by a single 3×3 Sobel-x convolution (9 pixel values $\{1, \dots, 9\}$, weights $\{-1, 0, 1, -2, 0, 2, -1, 0, 1\}$) and verified the accumulated output equals 8. This confirmed correct signed 8×8 multiplication, correct accumulation over 9 taps, and the done signal assertion timing.

Module test (M2/M3, compute_core, SystemVerilog TB). `tb_compute_core.sv` ran two extended test cases: T1 applied a 3×3 Sobel-x kernel (27 taps across 3 channels) and verified the accumulated output equals 24; T2 applied a deterministic mixed-sign 27-tap stream with values spanning the full INT8 range and verified the output equals 32,853. Both outputs were computed independently in Python using the same signed arithmetic.

Interface test (M2/M3, tb_interface.sv). Verified the complete AXI4-Lite protocol: AWVALID/AWREADY/WVALID/WREADY/BVALID/BREADY handshake, BRESP=OKAY on all writes, BUSY status bit assertion during computation, DONE bit assertion and RESULT register valid after computation. Also verified the AXI4-Stream 27-beat input handshake and correct TDATA/TLAST on the output.

Co-simulation (M3, tb_top.sv). End-to-end test of the complete top module: loaded Sobel-x weights via AXI4-Lite, streamed 27 input pixels via AXI4-Stream, verified output beat with TDATA=24 and TLAST=1, also verified the RESULT register via AXI4-Lite read. PASS confirmed.

Full-layer test (M4, tb_top.sv). The final testbench streams all 12,288 pixels of a $64 \times 64 \times 3$ synthetic image (pixel value = $(\text{row} \times 64 + \text{col} + 1 + \text{ch} \times 10) \% 128$), loads Sobel-x weights for filter 0 via AXI4-Lite (filters 1-15 use zero weights from reset), and verifies three specific output positions against independently computed Python reference values:

- `pos(0,0): hw=1, sw=1 ✓ — acc=300 (padding + Sobel-x on first row), >>8=1`
- `pos(0,1): hw=0, sw=0 ✓ — acc=18, >>8=0`
- `pos(1,0): hw=2, sw=2 ✓ — acc=528, >>8=2`

Total beats: 4,096/4,096. TLAST=1. Errors=0. **tb_top: PASS** (Verilator 5.032, wall time 0.107 s). The simulation waveform is shown in Figure 3 (`sim/final_waveform.png`), captured from a real VCD dump using GTKWave. The waveform shows the complete 135,655-cycle transaction: AXI write phase (Region A), pixel streaming with backpressure (Region B), and output beat emission with TLAST assertion (Region C).

DRC. KLayout DRC against sky130A rules checked 219 design rules and found **0 violations**.

7. Synthesis Results

Tool chain: OpenLane 2.3.10, Yosys 0.52, OpenROAD 2.0, PDK sky130A / sky130_fd_sc_hd, clock period 10 ns (100 MHz). The complete OpenLane run log is at synth/openlane_run.log.

Area. Yosys synthesis produced a netlist with **67,785 standard cells** and a pre-PnR cell area of 792,678 μm^2 . After place-and-route with OpenROAD, the final areas are:

- Standard cell instance area: 958,252 μm^2
- Core area: 1,757,930 μm^2
- Die area: 1,805,290 μm^2

The dominant area contributor is the sequential element count. Yosys reports 10,058 instances of sky130_fd_sc_hd_dfxtpt_2 (D flip-flop with enable). This is consistent with the design's register file: 16 filters $\times$ 27 taps $\times$ 8 bits = 3,456 weight register bits, plus 16 $\times$ 32-bit accumulators = 512 accumulator bits, plus the control FSM and line buffer state. The full area breakdown is in synth/area_report.txt.

Timing. Post-PnR static timing analysis was run across 9 corners (3 PVT corners $\times$ 3 min/nom/max operating conditions). Key results from synth/timing_report.txt:

- nom_tt_025C_1v80 (nominal): Setup WNS = -2.546 ns, Hold WNS = -0.140 ns
- nom_ff_n40C_1v95 (fast corner): Setup WNS = +1.898 ns ✓
PASSES
- nom_ss_100C_1v60 (slow corner): Setup WNS = -13.640 ns
- Worst overall: max_ss_100C_1v60, Setup WNS = -14.324 ns

The design does not close timing at 100 MHz in the nominal corner. The critical path runs through the 27-tap MAC accumulator chain in compute_core.sv. At the fast corner, the design passes with 1.898 ns of positive slack, indicating the logic depth is manageable and the failing corners are primarily due to high cell delays at low voltage and high temperature. To achieve timing closure at 100 MHz: (1) add a pipeline register at the accumulator output, reducing the critical path depth from 27 stages to two 13-14 stage halves, or (2) reduce the clock target to ~ 75 MHz where all corners pass.

Power. From OpenROAD report_power at nom_tt_025C_1v80 (synth/power_report.txt):

- Sequential: 40.14 mW (49.5%) — dominated by the large weight register file and accumulator flip-flops
- Combinational: 15.26 mW (18.8%) — MAC logic, mux trees, window decode
- Clock: 25.64 mW (31.6%) — clock tree driving 10,058+ flip-flops
- **Total: 81.04 mW**

Clock power is unusually high (31.6%) because the clock tree must drive over 10,000 flip-flops. Reducing the register count (e.g., by using SRAM macros for the weight file instead of flip-flop arrays) would substantially reduce both sequential and clock power.

8. Benchmark Results

Measurement method. The accelerator runtime is derived from the Verilator simulation cycle count. The testbench `tb_top.sv` records the cycle number when the 4,096th output beat is received (`cyc=135,655`). At a 100 MHz clock (10 ns period), this gives a runtime of $135,655 \times 10 \text{ ns} = \mathbf{1,356.55 \mu\text{s per inference}}$. The software baseline runtime of 1,257.30 μs is the median of N=1000 Python/NumPy measurements from M1 profiling. All raw numbers are in `bench/benchmark_data.csv`.

Results summary:

Metric	SW Baseline (M1)	M4 Accelerator	Ratio
Platform	AMD Ryzen 7 7730U	sky130A ASIC @ 100 MHz	—
Runtime/inference	1,257.30 μs	1,356.55 μs	0.927×
Throughput	795 inf/sec	737 inf/sec	0.927×
Performance	2.815 GFLOP/s	2.606 GFLOP/s	0.927×
Power	~15,000 mW (TDP)	81.04 mW	185× less
Energy/inference	~18,855 μJ	~109.96 μJ	171× less

Speedup = SW runtime / HW runtime = 1,257.30 / 1,356.55 = **0.927×** (**7% slower in throughput**).

The M4 accelerator does not achieve a throughput speedup over the CPU baseline at 100 MHz. The M1 projection assumed 3.2 GFLOP/s with perfect MAC utilization. The measured 2.606 GFLOP/s reflects two factors. First, FSM overhead: each output position requires WIN_LATCH (1 cycle) + MAC_RUN (27 cycles) + ACC_SETTLE (1 cycle) + OUT_PIPE (1 cycle) = 30 cycles minimum, giving a theoretical MAC utilization of $27/30 = 90\%$. Second, timing violations: the design does not close timing at 100 MHz in the nominal corner (WNS=-2.546 ns), meaning the true achievable frequency is approximately 75 MHz, which would give a runtime of ~1,808 μs — 1.44× slower than the CPU.

The primary contribution of the M4 design is energy efficiency: 171× lower energy per inference. At 81.04 mW and 1,357 μs per inference, the accelerator uses 109.96 μJ per inference, compared to the CPU's estimated 18,855 μJ (15W TDP $\times$ 1,257 μs). For edge deployment applications where battery life is the binding constraint, this 171× energy reduction is the critical metric.

9. What Did Not Work

1. Timing closure at 100 MHz. The design fails setup timing at the nominal process corner (WNS=-2.546 ns at `nom_tt_025C_1v80`). The root cause is the deep combinational chain through the 27-tap MAC accumulator: each of the 27 accumulation steps adds one 8-bit signed adder delay, and the total chain depth exceeds the 10 ns clock period. The fast corner passes cleanly (+1.898 ns), confirming the logic structure is sound. Three remedies were not implemented due to time constraints: (1) adding a pipeline register at the accumulator midpoint (splitting 27 stages into two 13-14 stage halves), (2) reducing the

clock period to 13 ns (~77 MHz) where the nominal corner clears, and (3) providing a custom SDC file with false path annotations for the reset network and multicycle path declarations for the weight register load path.

2. Full OpenLane flow did not complete. The OpenLane run reached Stage 55 of 76 (post-PnR STA) before stopping. Stages 57-76 — including Magic GDSII stream-out, write LEF, XOR comparison, and LVS — failed due to a version incompatibility between Magic 8.3.105 (the version available from the Ubuntu package repository) and the version of the sky130A PDK installed by volare (bdc9412b). Magic 8.3.105 does not recognize the device model syntax used in the PDK's sky130A.tech file (specifically the device keyword sections on lines 5880-5900). As a workaround, KLayout was used for both GDSII stream-out (producing top.klayout.gds) and DRC (producing drc_report.xml with 0 violations across 219 rules). LVS could not be completed without the Magic SPICE extraction. The correct fix would be to compile Magic from source at the version pinned by the OpenLane volare PDK installation.

3. Throughput did not exceed the CPU baseline. The M1 analysis projected a $2.5\times$ speedup based on the theoretical peak throughput. The measured result is $0.93\times$ — the accelerator is 7% slower in throughput. Contributing factors: (a) FSM overhead reduces MAC utilization from the theoretical 100% to ~90%; (b) timing violations mean the design cannot reliably operate at 100 MHz; (c) the CPU baseline achieved a higher-than-expected 2.815 GFLOP/s because NumPy uses compiler-optimized inner loops for this specific convolution pattern. A redesign with a fully pipelined dataflow (new window loading during the current MAC run) would eliminate the WIN LATCH and FLUSH state overhead, improving effective throughput by approximately 10%.

4. What I would do differently. First, use the pre-built Docker image openroad/ubuntu22.04-dev rather than manually installing and patching individual EDA tools; version mismatches consumed the majority of the implementation time for this milestone. Second, write the design to a 75 MHz clock target from the start, with an SDC file that properly constrains the reset path and declares multicycle paths for the weight registers. Third, implement the MAC array with a pipeline register at the accumulator output to enable 100 MHz operation. Fourth, replace the weight flip-flop array with a small SRAM macro to reduce area and power by approximately 30%.